Software Engineering

Dr. Fatma ElSayed

Dr. Ahmed Yousry

Computer Science Department

Course Information

Contents

- Introduction to Software Engineering
- Software Processes
- Requirements Engineering
- System Modelling Part1
- System Modelling Part2
- System Architecture
- Software Testing Strategies
- Software Testing Techniques
- Technical Metrics for Software

Course Information

Contents

- Introduction to Software Engineering
- Software Processes
- Requirements Engineering
- System Modelling Part1
- System Modelling Part2
- System Architecture
- Software Testing Strategies
- Software Testing Techniques
- Technical Metrics for Software

System Modeling

- System modeling is the process of developing ***abstract models*** of a system, with each model presenting *a different view* or perspective of that system.

- System modelling has generally come to mean representing the system using some kind of ***graphical notation***, which is now almost always based on notations in the ***Unified Modelling Language (UML).***

When to use:

- During the requirements engineering process: Models are used to help derive the requirements for a system.
- During the design process: Models are used to describe the system to engineers implementing the system.
- After implementation: to document the system's structure and operation.

UML

- The Unified modeling Language is a set of ***13 different diagram types*** that may be used to model software systems.

Structural Diagrams

1. Class Diagram
2. Object Diagram
3. Component Diagram
4. Composite Structure Diagram
5. Deployment Diagram
6. Package Diagram

Behavioral Diagrams

1. Use Case Diagram
2. Sequence Diagram
3. Activity Diagram
4. State Machine Diagram
5. Communication Diagram
6. Interaction Overview Diagram
7. Timing Diagram

UML

- The Unified modeling Language is a set of ***13 different diagram types*** that may be used to model software systems.

Structural Diagrams

1. Class Diagram
2. Object Diagram
3. Component Diagram
4. Composite Structure Diagram
5. Deployment Diagram
6. Package Diagram

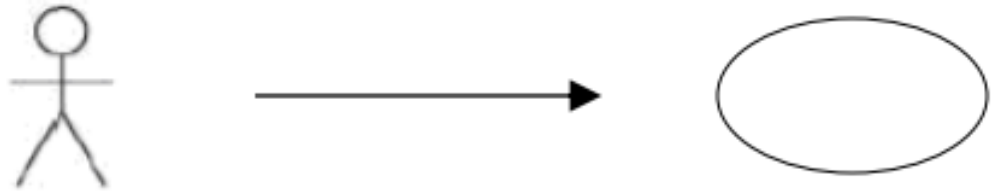
Behavioral Diagrams

1. Use Case Diagram
2. Sequence Diagram
3. Activity Diagram
4. State Machine Diagram
5. Communication Diagram
6. Interaction Overview Diagram
7. Timing Diagram

Use Case Modeling

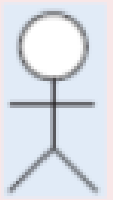
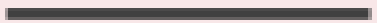
Use Case Modeling

- Use case modeling is widely used to support ***requirements elicitation*** because it describes what a user expects from a system.
- Acting as a **discussion tool** between developer and user, and offer a common language for agreeing on the **functions** of the software system.
- Use case diagrams show the interactions between:
  - **Use cases:** what the system should do.
  - **Actors:** anyone or anything outside the system's scope but interacts with it with the system (Individual, group, company,…).



Use Case Diagram: Elements & Symbols

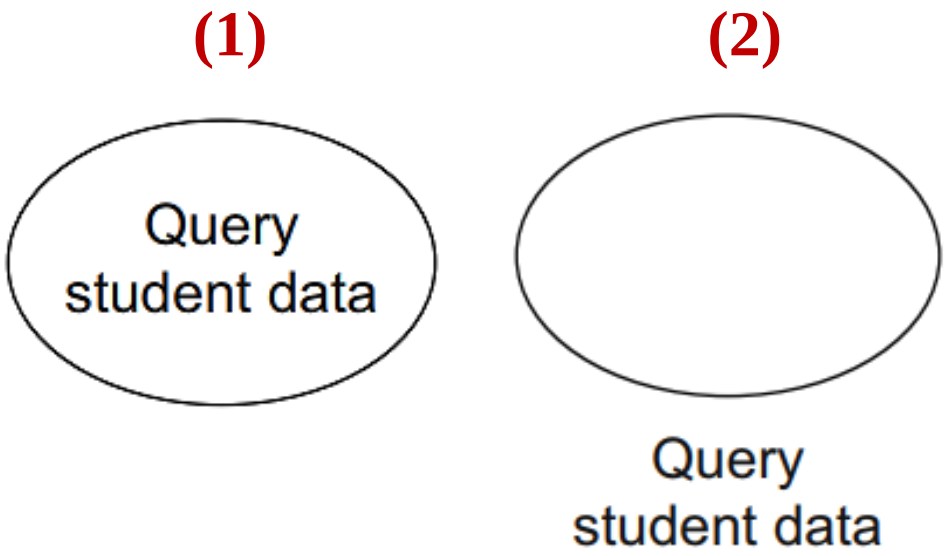
Introduces a means of defining what a proposed system should do from a **user's perspective**.

The actors, represented by **stick figures**;	
The use cases, represented by **ovals**, and are used to represent tasks.;	
The relationships between actors and use cases, represented by **lines**.	
Note symbol: is used to clarify an aspect or a task, it can be used with any UML diagram.	
A **dashed line** is used to attach the note to the model element to which it refers.	

Use Case Diagram

Alternatives for use cases:

- The alternatives are all equally valid, but the first alternative ***(1)***, the ellipse that contains the name of the use case, is the one most commonly used.
- The use cases are generally grouped within a rectangle. This rectangle represents the boundaries of the system to be System described.



Use Case Diagram: Example

System: (what is being described?)

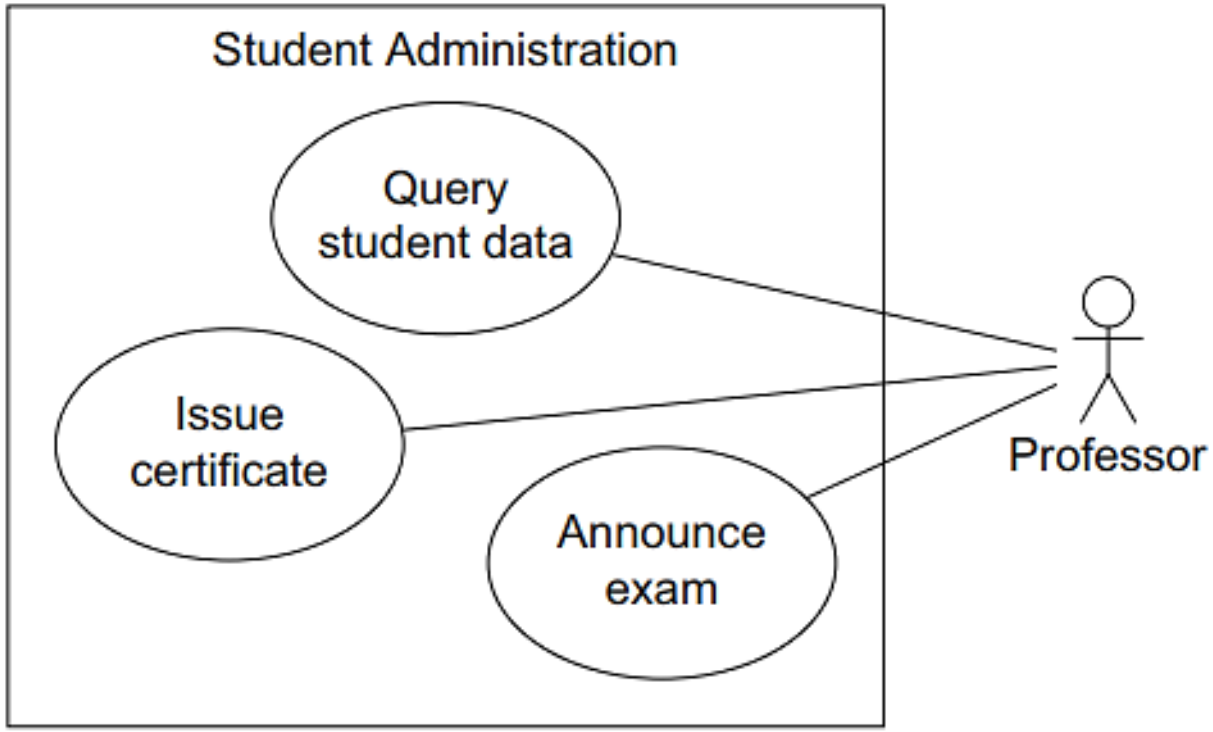
- Student administration system.

Actors: (who interacts with the system?)

- Professor.

Use Cases: (what can the actors do?)

- Query student data.
- Issue certificate.
- Announce exam.



Use Case Diagram System Boundary

The **system boundary** determines the scope of the system.

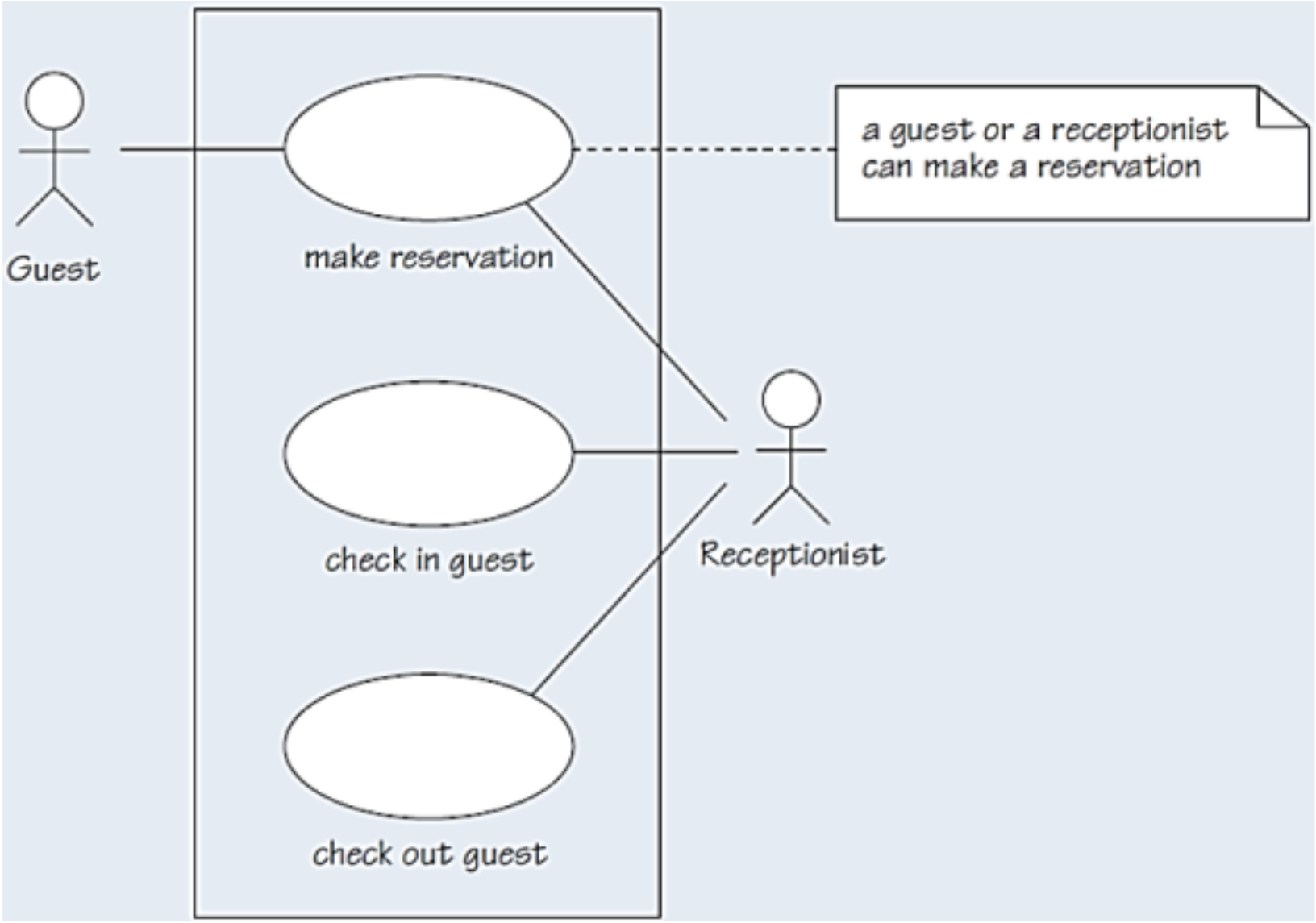
- It distinguishes between internal and external components.
- The *external* components are *actors* and the *internal* components are the *use cases*.
- In UML it is optional to use system boundary notation.
- A solid box drawn around the use cases with the actors located outside it represents the system boundary.
- **We use system boundary notation when the system is complex and includes several subsystems.**

Use Case Diagram: Example

A simple use case diagram for the main tasks in a hotel system:

- Making reservations,
- Checking in, and
- Checking out, where

The reservation can be done by the receptionist or a guest (by Phone/Web).



Use Case Diagram: Identifying the Actors

- Actors can be Human *(e.g., Student or Professor)* or Non-human *(e.g., E-mail Server)*.

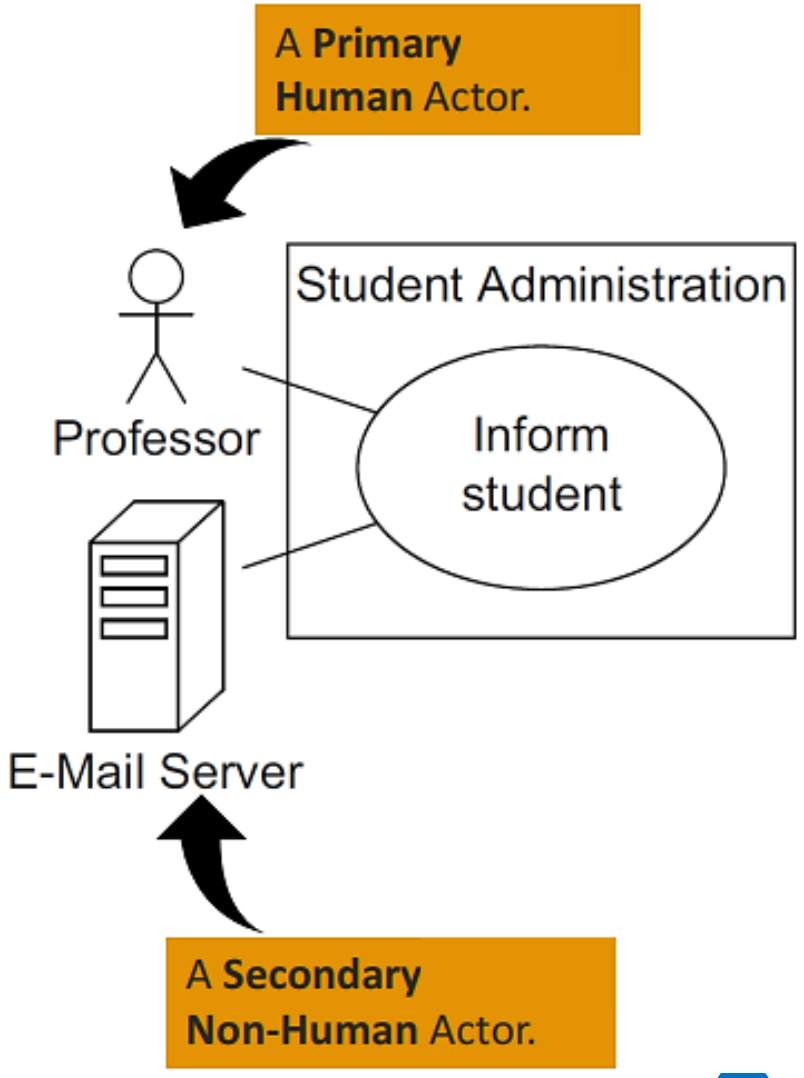
A Primary actor:

- *Initiates* the execution of use cases,
- Takes an actual *benefit* from the execution of the use case, whereas

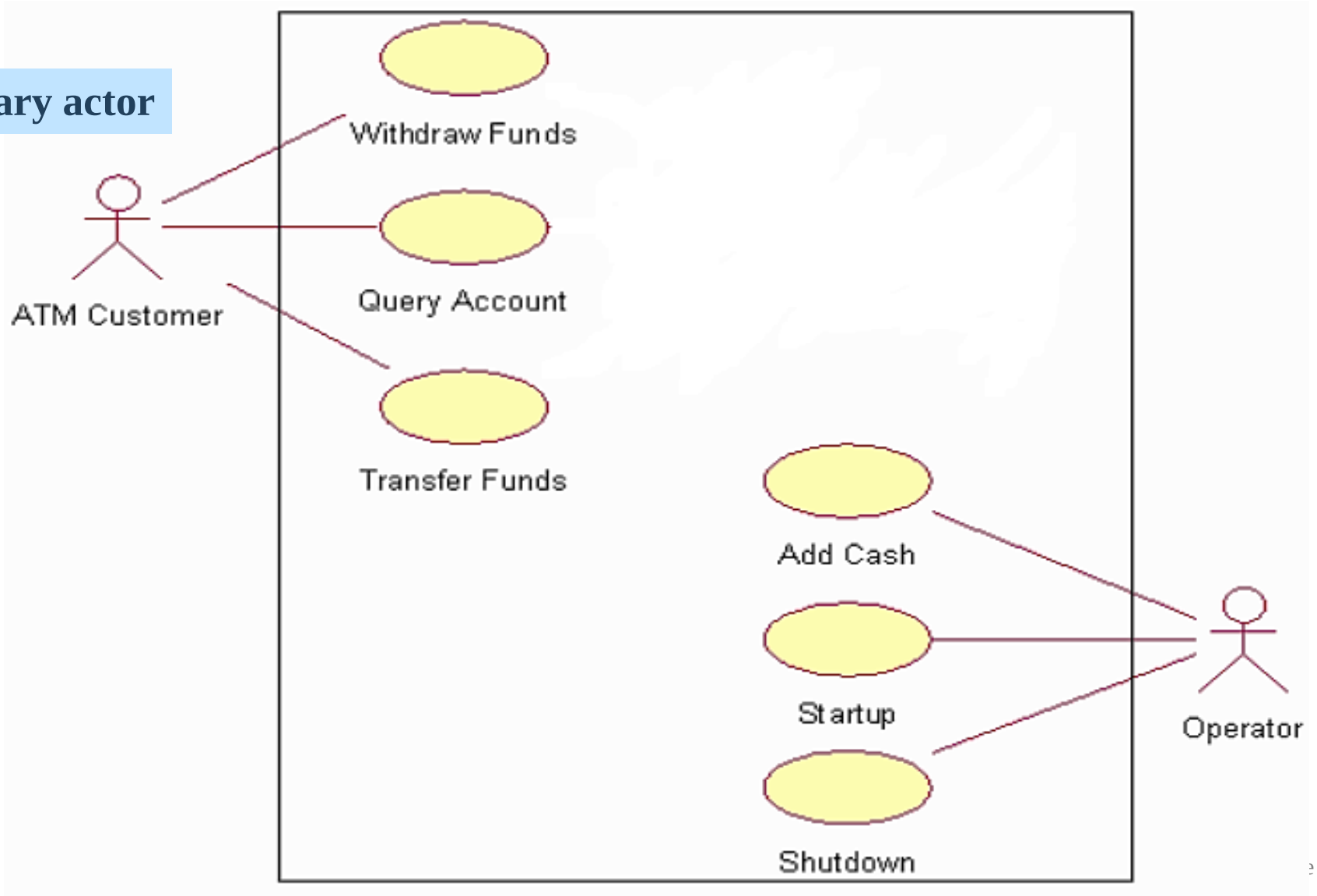
A Secondary actor:

- *Helps* the primary actor to complete the use case,
- Receives *no direct* benefit from the execution of the use case.

Who starts the use case? **(→ Primary)**
Who assists or provides a service? **(→ Secondary)**



Use Case Diagram: Example



Use Case Descriptions

- Use Case diagrams aren't intended to show in which **order** the use cases are executed.
- Use cases are an **implementation-independent**
- Use case diagrams give a fairly simple overview of an interaction, so you have to provide more detail to understand what is involved. This detail can either be:
  - A simple textual description *(flow of events)*
  - A structured description in a **table**.
  - A sequence diagram

Use Case Descriptions

Each use case description should contain the following parts in minimum:

- **Use case:** identifier and name.
- **Initiator:** name of the actor who initiates the process.
- **Pre-condition(s):** a condition that must hold before this use case can be carried out.
- **Post-condition(s):** a condition that must hold after the use case has been completed.
- **Main Success Scenario:** a single sequence of steps that describe the main success scenario. You can number the steps. A scenario is an instance of a use case.
- **Goal:** a short description of the goal of the use case;

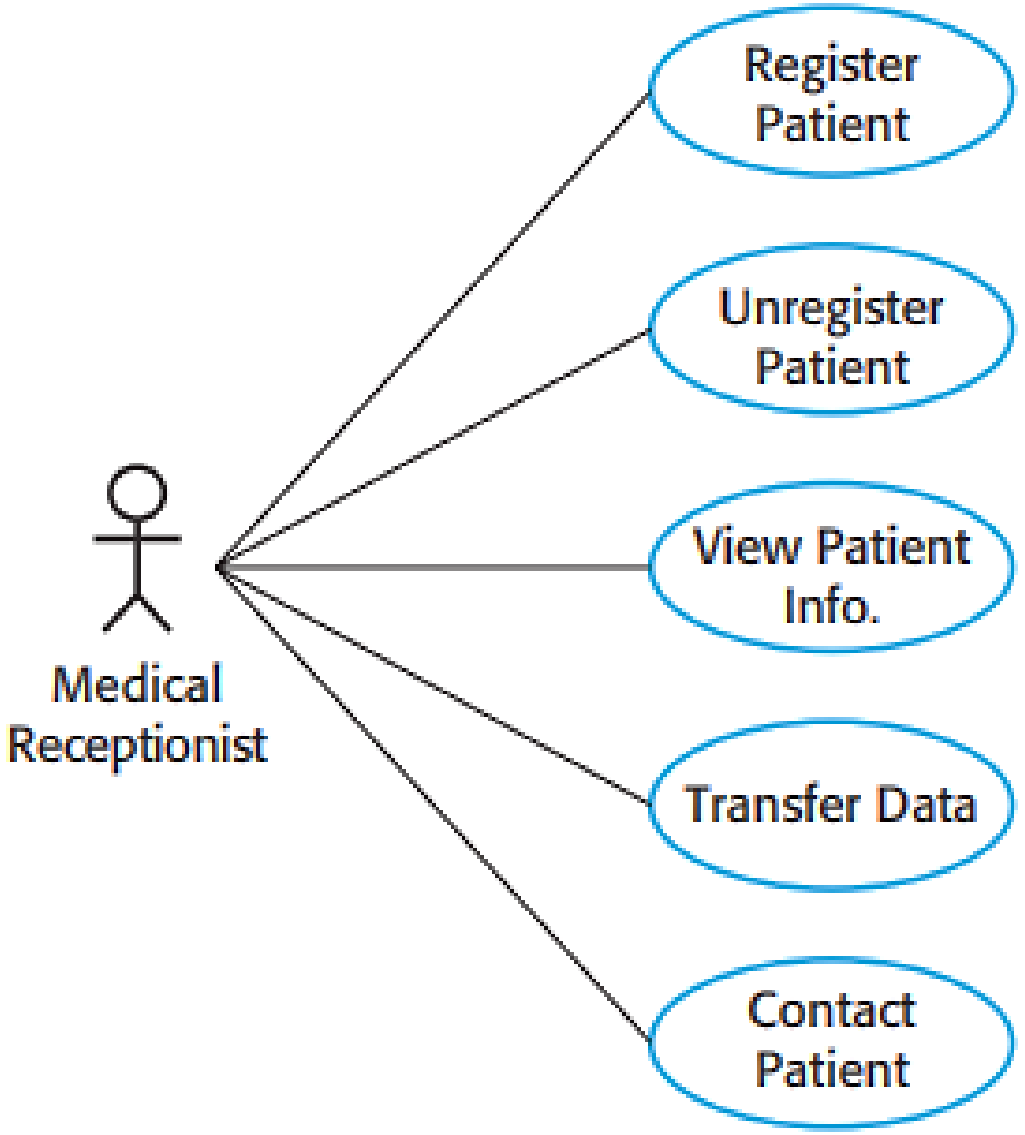
Table 1 A textual description of a use case in the hotel domain

Identifier and name	*make reservation*
Initiator	*Guest* or *Receptionist*
Goal	A room in a hotel is reserved for a guest.
Precondition	None (i.e. there are no conditions to be satisfied prior to carrying out this use case).
Postcondition	A room of the desired type will have been reserved for the guest for the requested period, and the room will no longer be free for that period.
Assumptions	The expected initiator is a guest (using a web browser to perform the use case) or a receptionist. The guest is not already known to the hotel's software system (see step 5).
Main success scenario	
1	The guest/receptionist chooses to make a reservation.
2	The guest/receptionist selects the desired hotel, dates and type of room.
3	The hotel system provides the availability and price for the request. (An offer is made.)
4	The guest/receptionist agrees to proceed with the offer.
5	The guest/receptionist provides identification and contact details for the hotel system's records.
6	The guest/receptionist provides payment details.
7	The hotel system creates a reservation and gives it an identifier.
8	The hotel system reveals the identifier to the guest/receptionist.
9	The hotel system creates a confirmation of the reservation and sends it to the guest.

Use Case Scenarios

- A scenario describes a sequence of interactions between the system and some actors. For each use case there is a set of possible *scenarios*. A scenario is an instance of a use case. Here are two examples of scenarios:
  - A member of a lending library wishes to borrow a book, and is allowed to do that as long as they have no outstanding loans.
  - Another member wishes to borrow a book, but has exceeded the quota for the number of books that can be borrowed.
- In each scenario the member wishes to borrow a book, but both the circumstances and outcomes of events are different in each instance.
- The main **success scenario** shows the steps normally followed to achieve the stated goal of the use case.

Use Case Diagram: Example

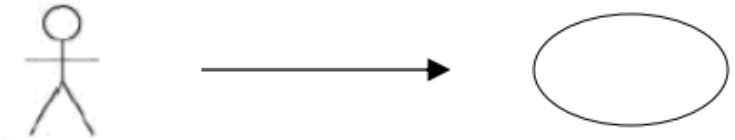


Use Case Relationships

Use Case Diagram: Relationships

1. Association

Used to show the relationship between a **use case** and an **actor** only



2. Inclusion
3. Extension

Only two relationships allowed between **use cases**

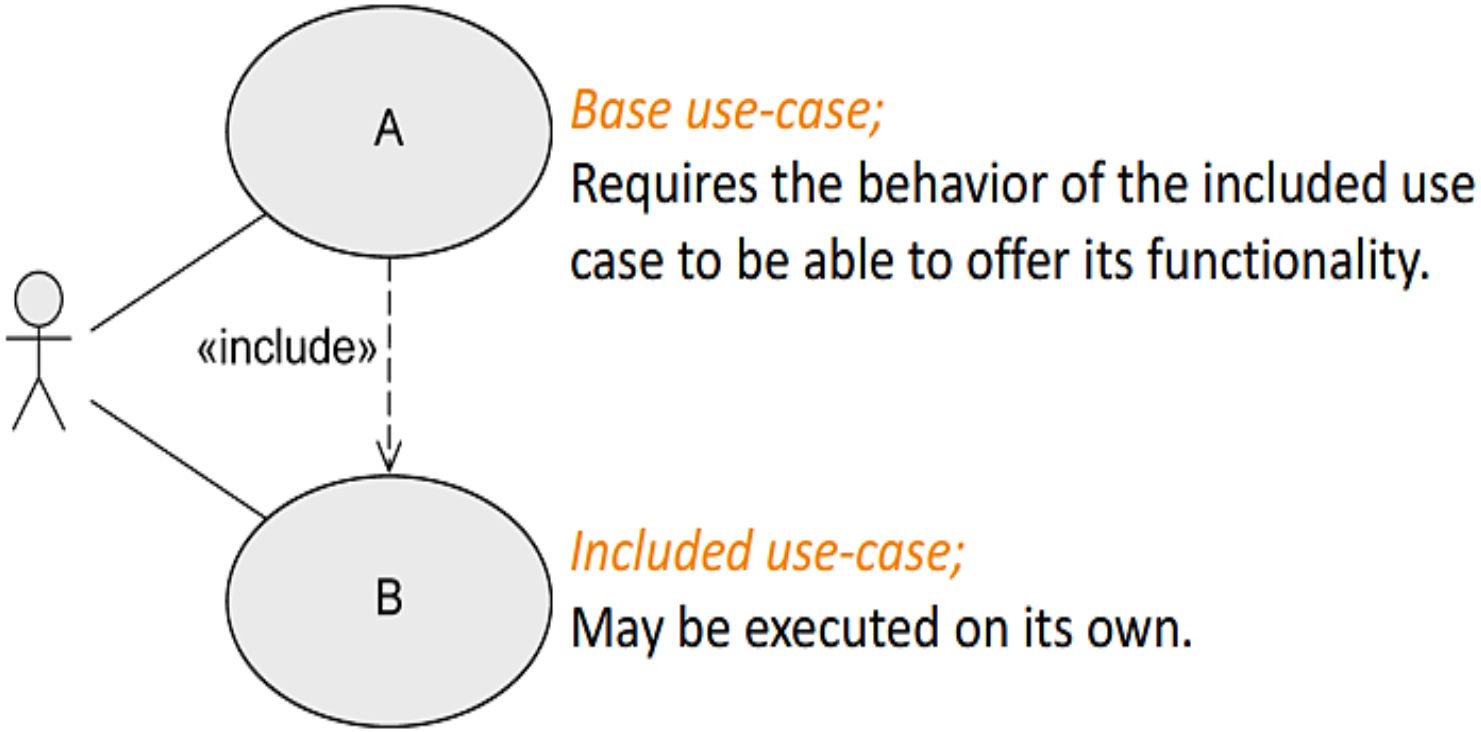
4. Generalization

Between **actors,** and sometimes between **use cases**

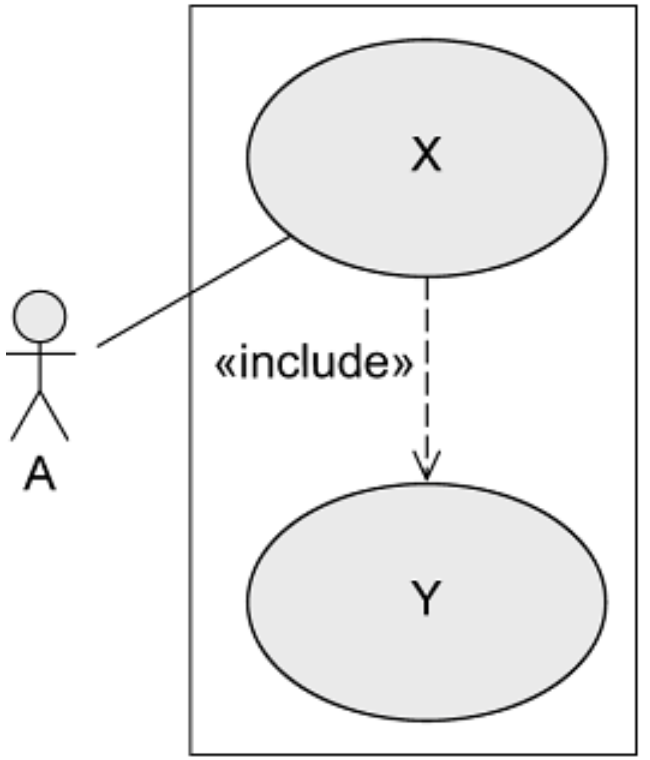
Inclusion Relationship

– the «include» – Relationship:

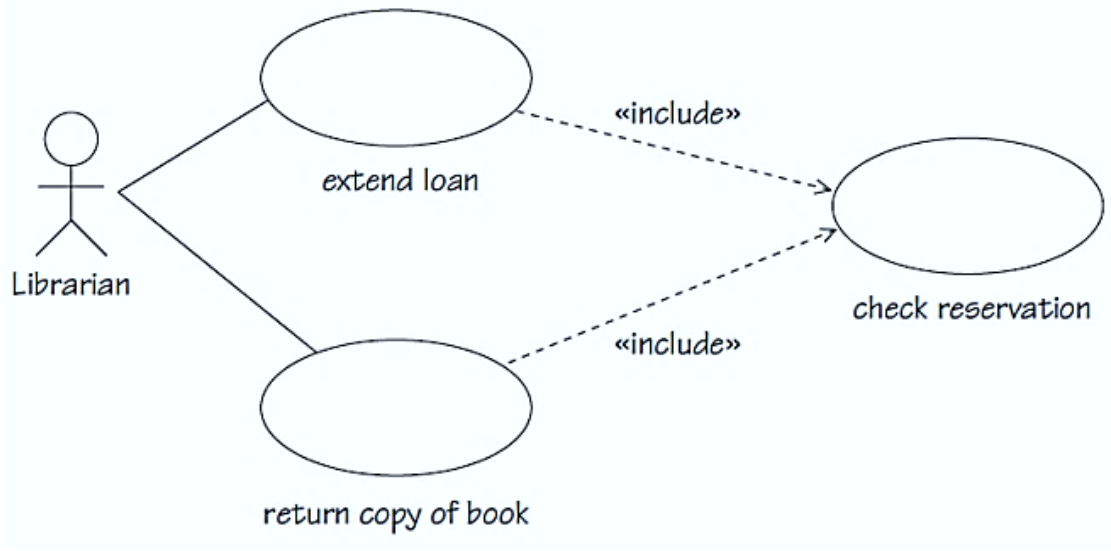
- The behavior of one use-case *(included use case)* is integrated in the behavior of another use case *(base use case)*.

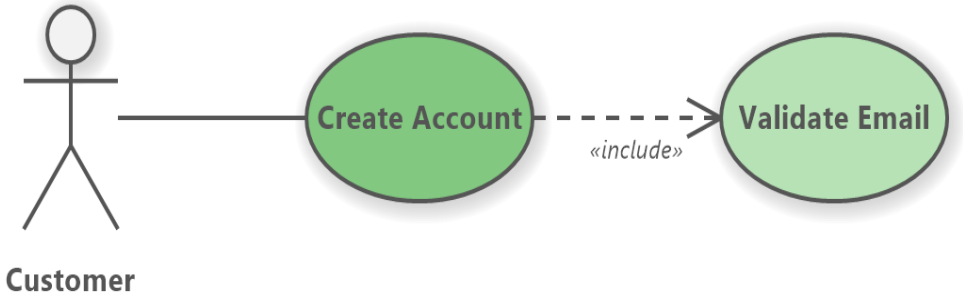


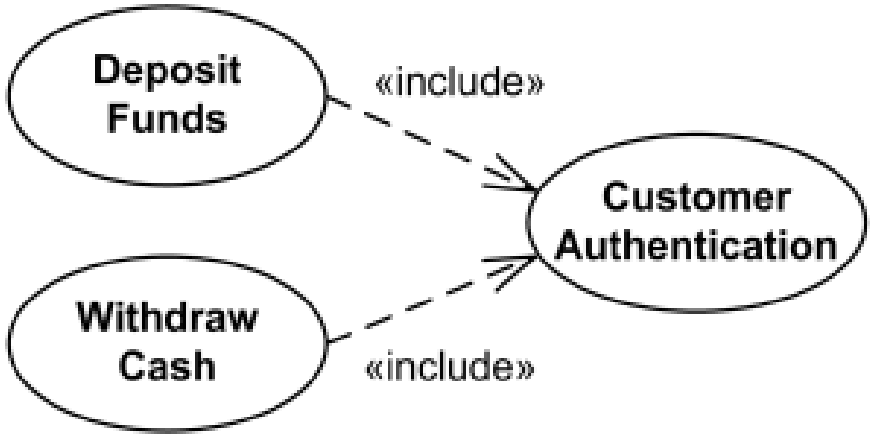
Best Practice

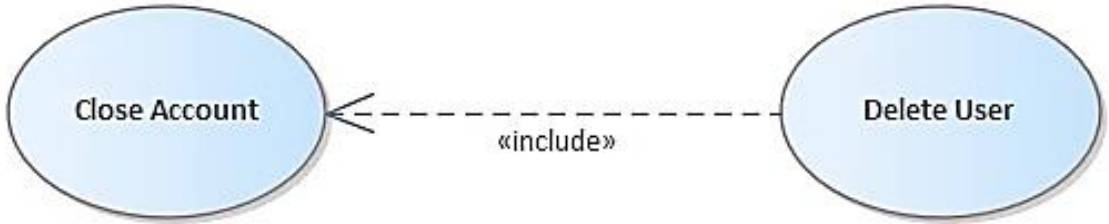


Inclusion Relationship: Examples



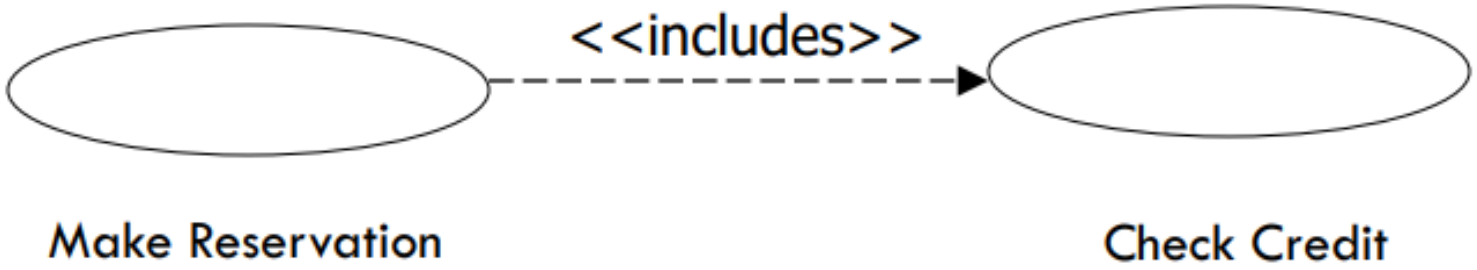






Inclusion Relationship

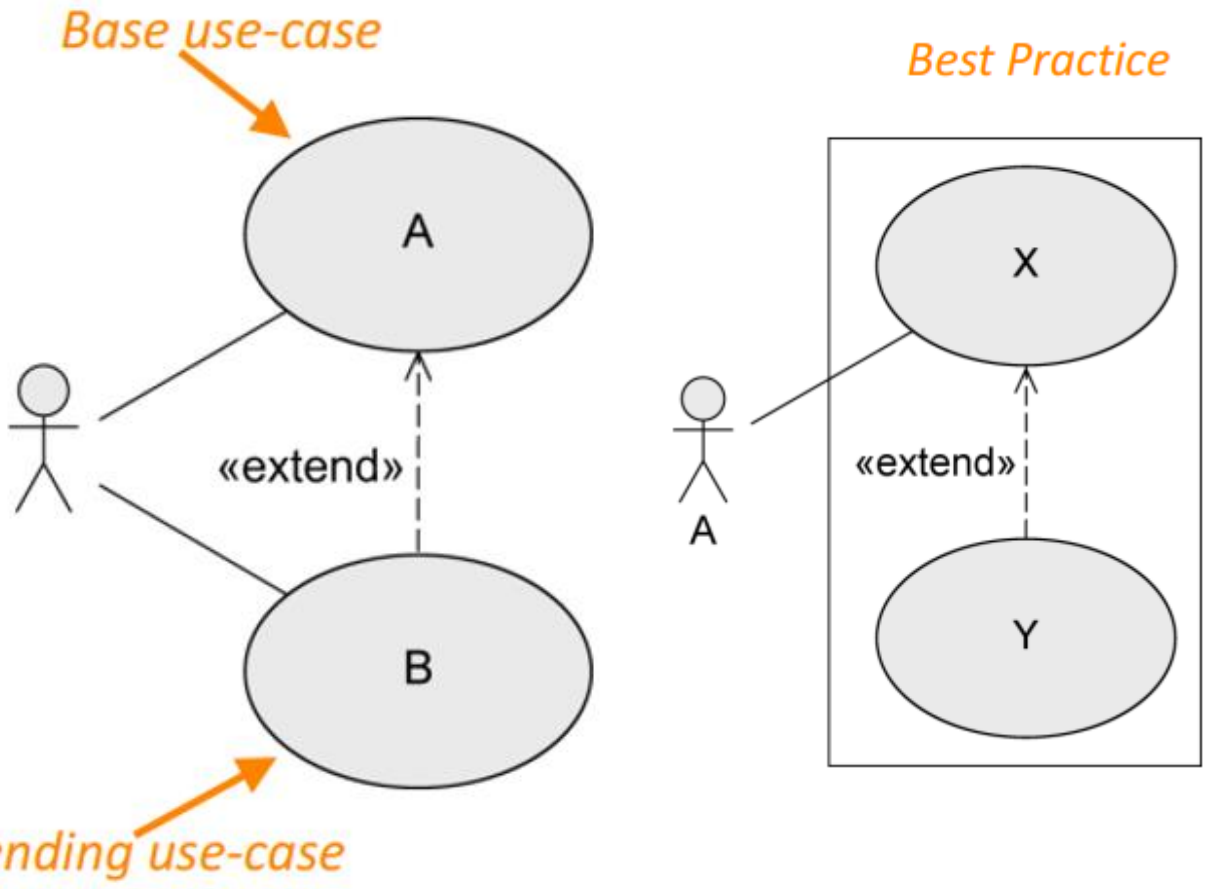
- In the process of eliciting and specifying requirements, you may find a certain amount of **common behavior** in two or more of your use cases.
  - You can record the shared behavior **in a new use case** and connect it to the use cases that it came from with a dashed arrow *(indicating a dependency relationship)* pointing from the original use case to the new one.
  - Hence the dependency arrow is labeled with the «include» stereotype.



Extension Relationship

– the «extend» – Relationship:

- Allows one use case *(extending)* to extend the functionality provided by another use case *(base)*.
- As the name implies it *extends the base use case and adds more functionality to the system.*



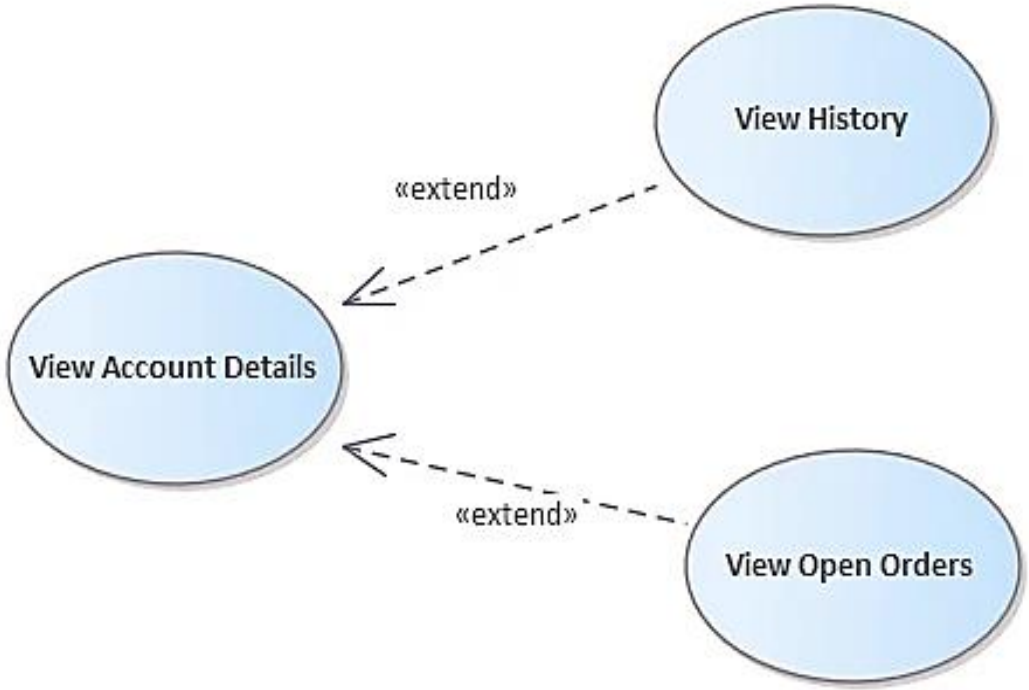
Extension Relationship

Few things to consider when using the **«extend»** relationship:

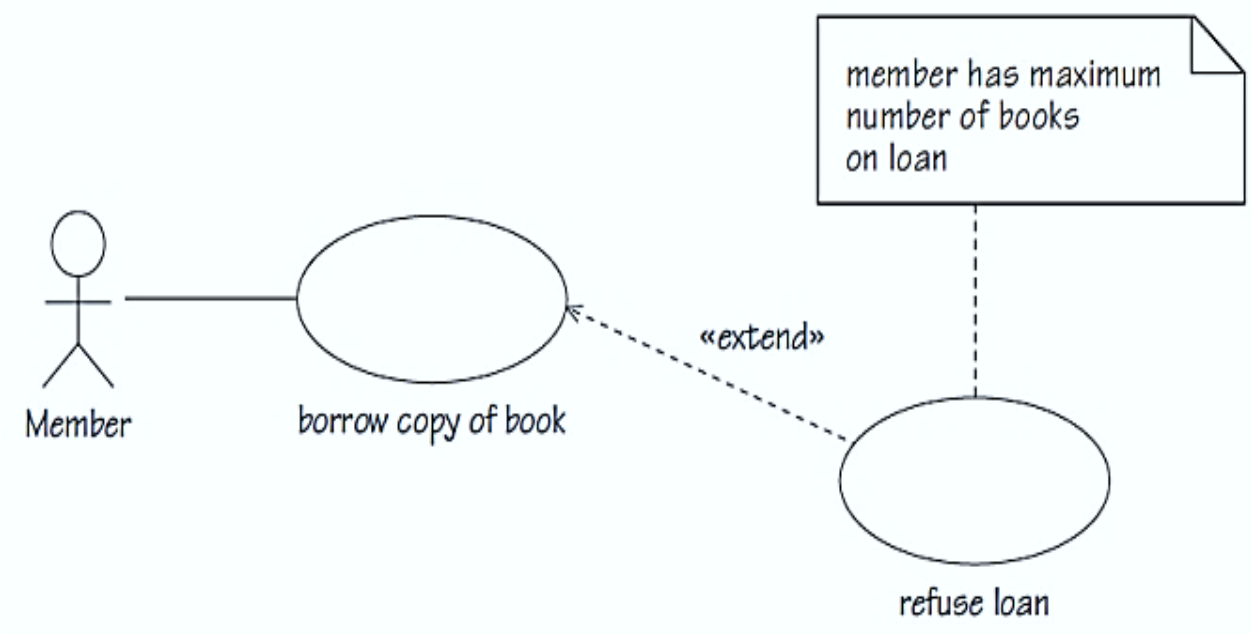
- The extending use case is dependent on the extended (base) use case.
- The extending use case is usually **optional**.
- The extended (base) use case must be meaningful on its own.

- **«Extend»** is a UML use case relationship that adds optional or conditional behavior to a base use case at defined extension points.
- You should add a **condition** to each extension – by a note to specify when the variant behavior will be included.

Extension Relationship: Examples

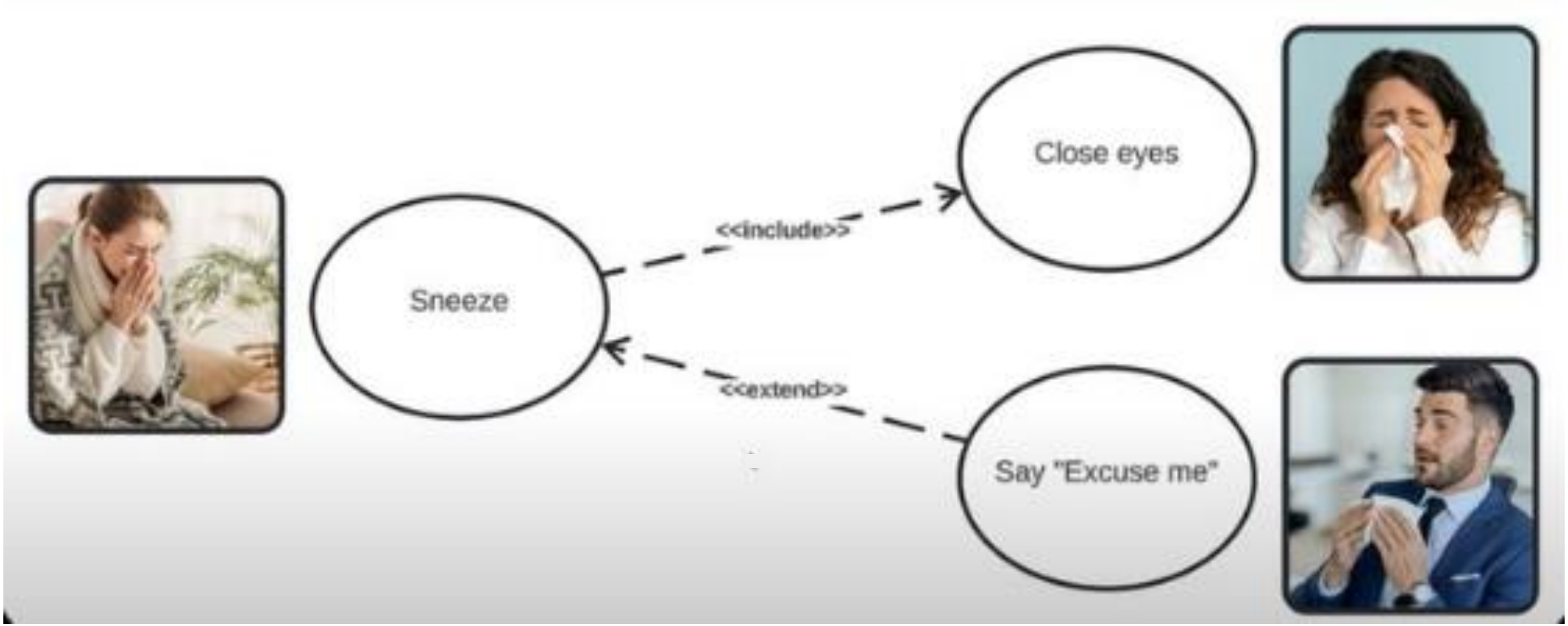


Optional Example



Conditional example

Inclusion & Extension Relationship



Generalization Relationship of Actors

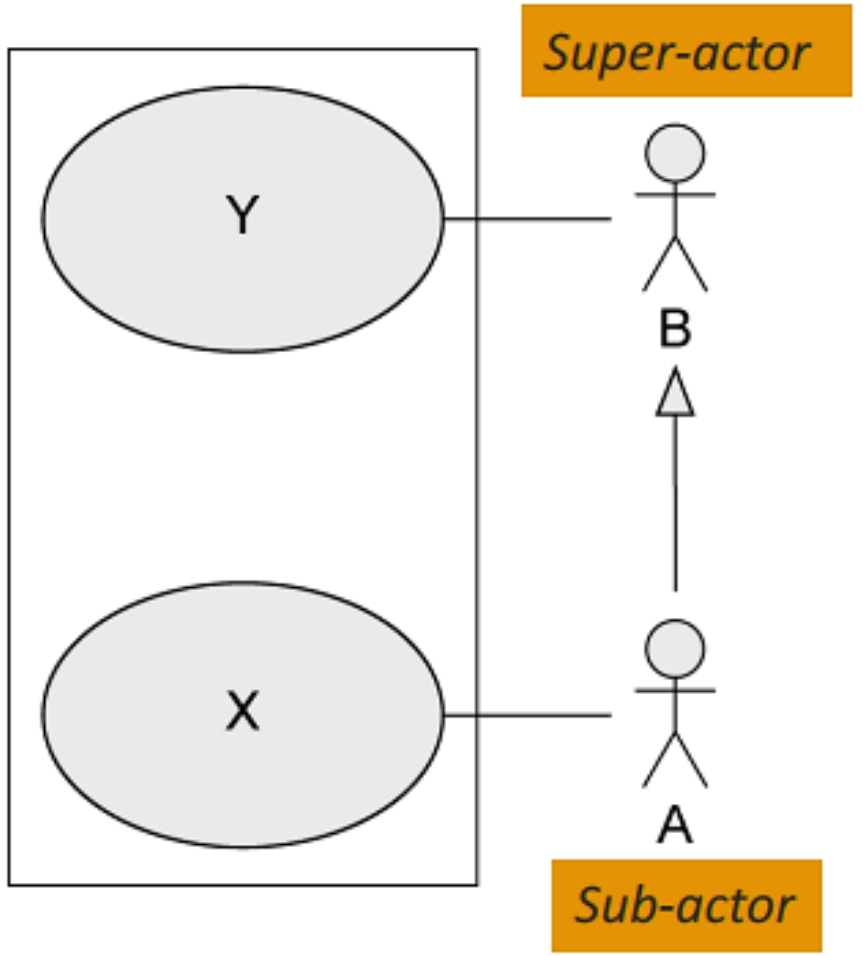
Actors may have common properties (using the same use cases).

E.g., It is possible that not only professors but also assistants (i.e., the entire research personnel) are permitted to view student data.

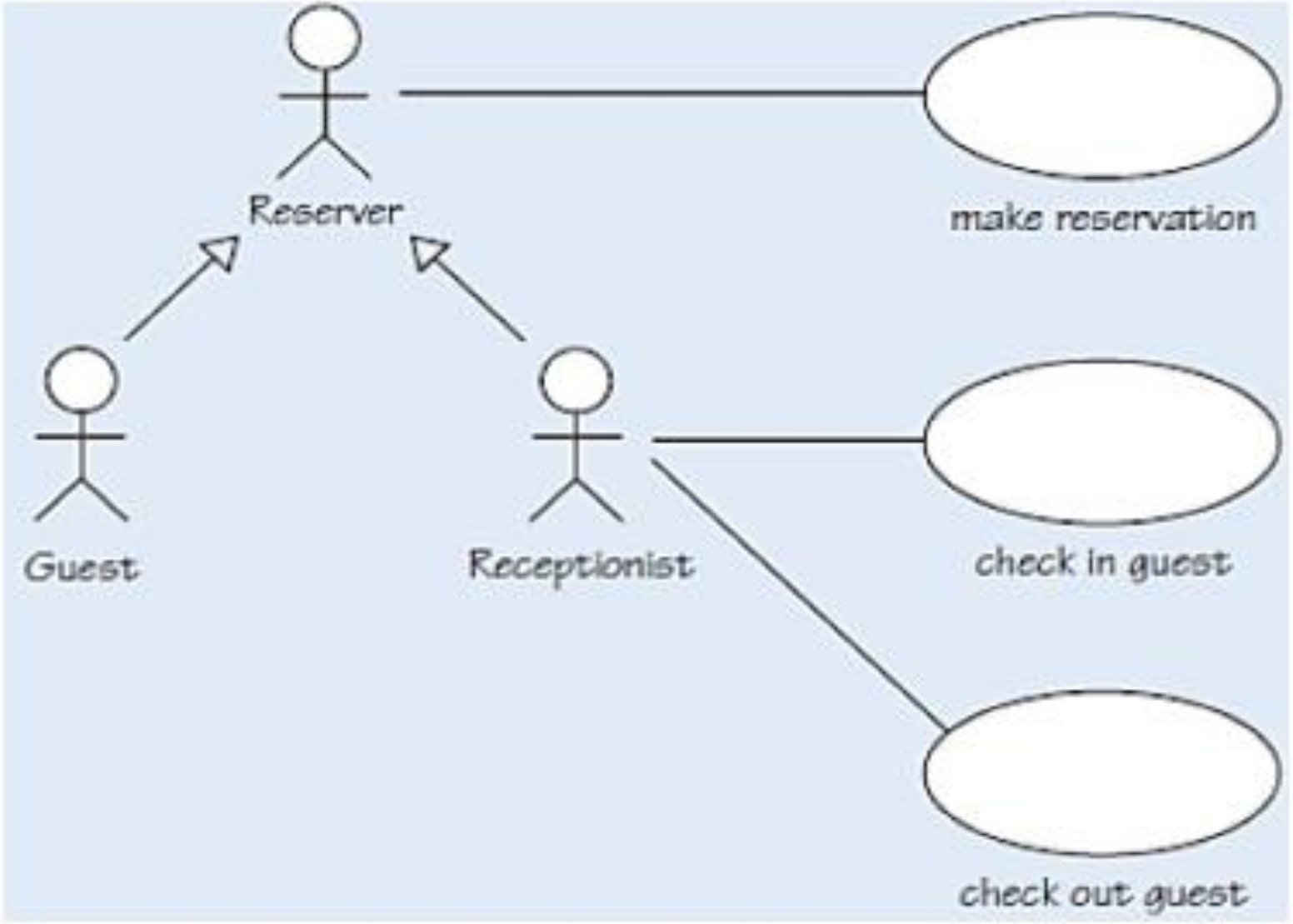
- To express this, actors may be involved in an **inheritance relationship *(generalization)*** with one another.
- When an actor Y ***(sub-actor)*** inherits from an actor X ***(super actor)***, Y is involved with all use cases with which X is involved.
- In simple terms, generalization expresses an **“is a”** relationship.

Generalization Relationship of Actors

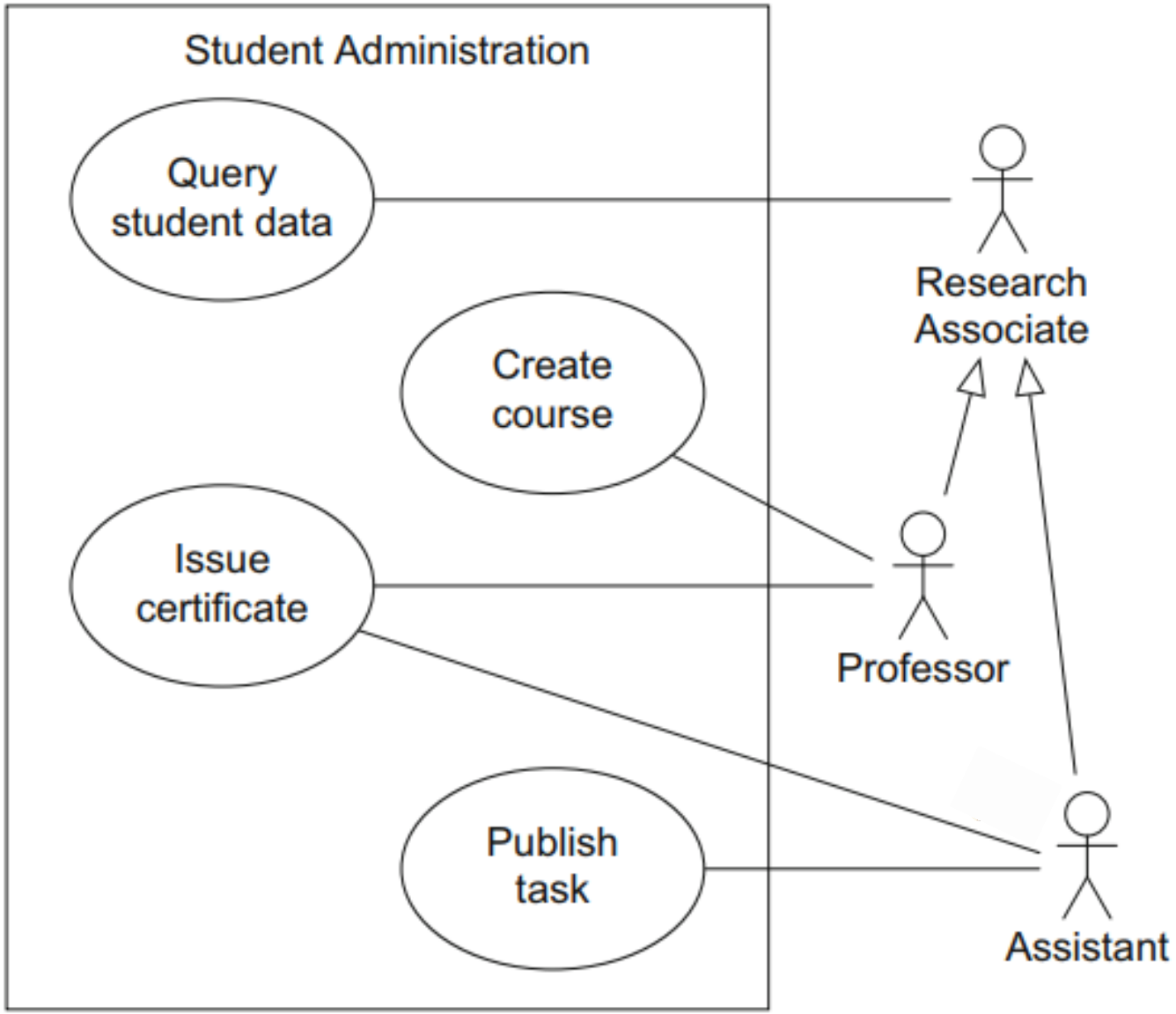
- Actor A inherits from actor B.
- Actor A can communicate with use-cases X and Y.
- Actor B can only communicate with the use-case Y.
- Multiple inheritance is allowed.



Generalization Relationship of Actors

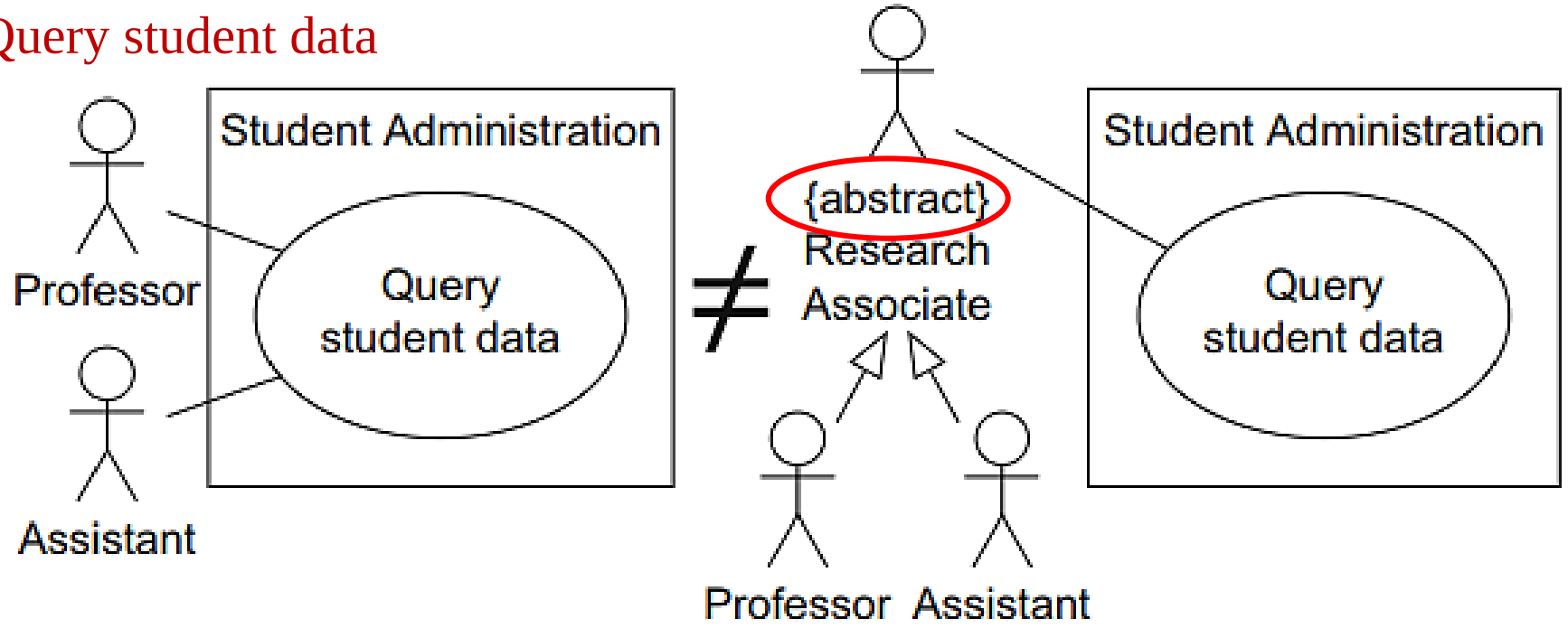


Generalization Relationship of Actors



Generalization Relationship of Actors

Professor **AND** Assistant needed for executing Query student data



{Abstract} actors

- Makes sense in the context of an inheritance relationship:
- The common properties of the sub-actors are grouped and described with abstract super actor.

Professor **OR** Assistant needed for executing Query student data

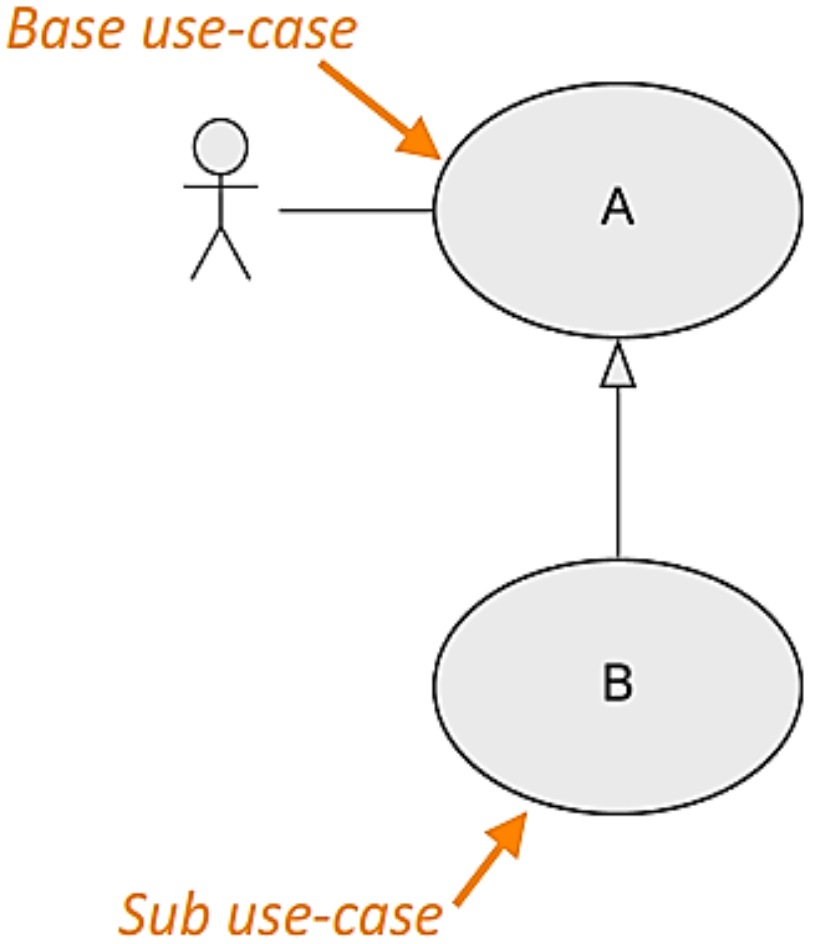
Generalization Relationship of Use cases

Generalization of Use Cases:

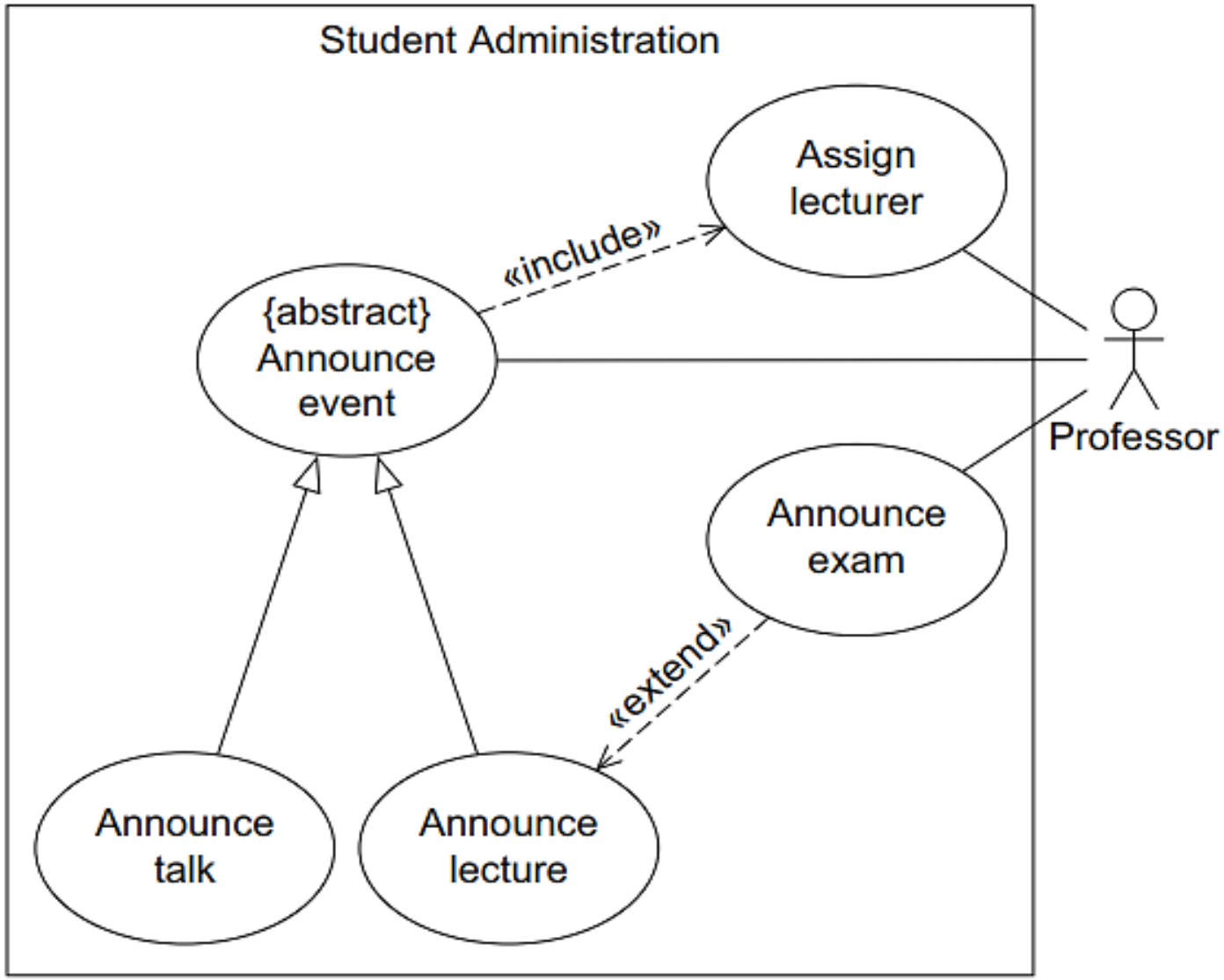
- Use case A generalizes use case B.
- B inherits the behavior of A and may either extend or overwrite it.
- B also inherits all relationships from A.

A may be labeled {abstract}

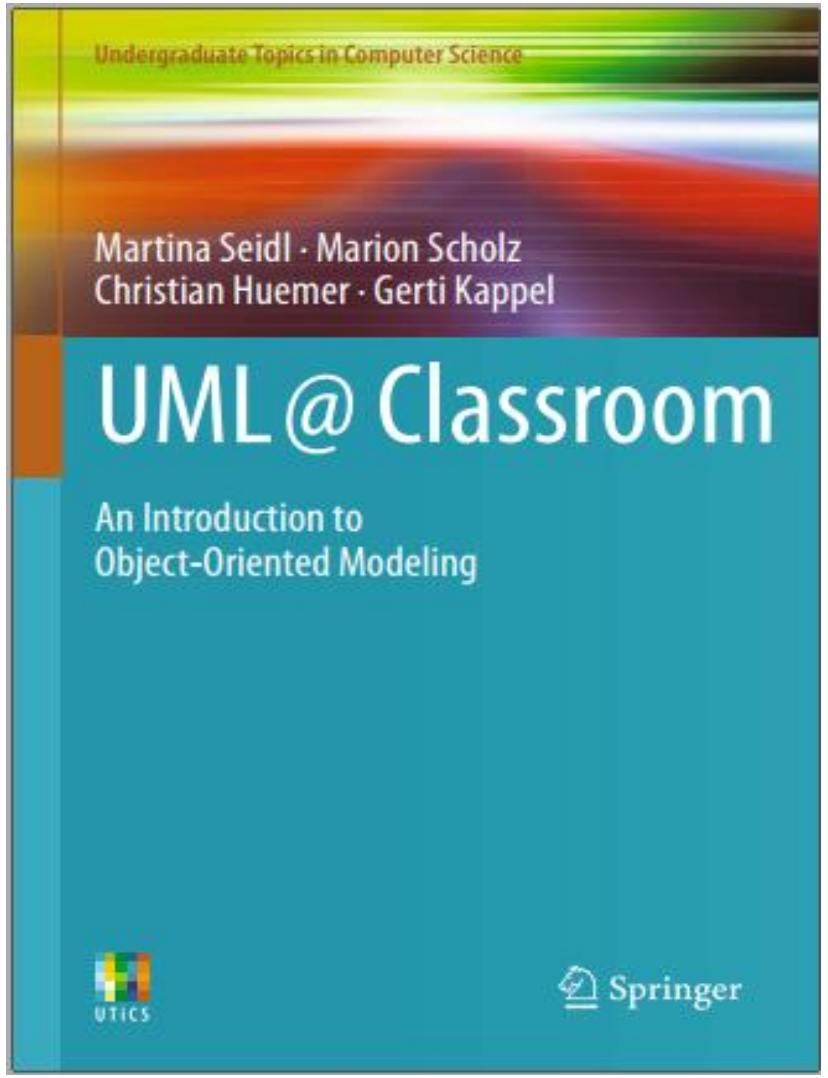
- Cannot be executed directly.
- Only B is executable.



Generalization Relationship of Use cases



Additional References



Thank You